How commands reach the motors

Two private applications, two buses and two bridges; only serialized frames cross between cores.

M7 · UI controller · 480 MHz

CLI + GUI
Commands and presentation

→

Motion client
Tracks requests and job history

↔

Planning
Conversions, paths and profiles

↕

Private pneumo bus ↔ ICC Bridge

One ICC coroutine context / worker (priority 4). All three lanes dispatch here.

M4 · IO controller · 240 MHz

Controller
Admission and ownership

→

Executor
Profiles → pulse schedules

→

TIM2 + DMA
Three physical STEP outputs

↕

Private pneumo bus ↔ ICC Bridge

One ICC coroutine context / worker (priority 4). Execution runs in separate workers.

↕ encode / decode

↕ encode / decode

Shared SRAM4 · three independent queues in each direction

6 queues total

URGENT
Stop, cancel, lease and link control
Up to 2,048 bytes per frame
▢▢▢▢ 4 slots per direction

NORMAL
Preparation, actions, replies and state
Up to 2,048 bytes per frame
▢▢▢▢ 4 slots per direction

BACKGROUND
IO logs → UI → USB console
Up to 512 bytes per frame
▢▢▢▢ 4 slots per direction

HSEM is the doorbell: it wakes a notification worker. The message bytes remain in shared memory.

What crosses the bridge

Services	request → deferred reply
Actions	acceptance → progress → terminal result
Topics	latest motor state at 20 Hz; bounded logs

Routes are declared in common/icc/catalog/{services,actions,topics}.hpp.

Separate budgets, shared dispatch

Urgent traffic has its own capacity, but it still needs CPU time in the same ICC context.

UI lease: renew every 50 ms · expires after 250 ms

M4 execution: 20 ms maintenance · 40 ms commitment

Memory: storage, execution and working space

Logical allocations, not to scale. RAM usage below belongs to the last flashed test image.

M4 private RAM

D2 SRAM · 288 KiB total

Code alias 0x10000000 = physical 0x30000000

2 KiB vectors · 0x10000000

192 KiB fast code + constants

0x10000800 · 195,280 B linked

86 KiB data + DMA buffers

0x30030800 · 84,640 B linked

8 KiB interrupt stack · 0x30046000

AXI heap region · 256 KiB

0x24040000 → 0x24080000

Unwind metadata precedes the heap.
Actual allocator capacity: 248,920 B.

The SRAM alias is not additional memory.

M7 private RAM

ITCM · 64 KiB

0x00000000 · 16,088 B linked

DTCM · 128 KiB

0x20000000: vectors, 2 KiB
0x20000800: data, 110 KiB
0x2001C000: gap, 8 KiB
0x2001E000: interrupt stack, 8 KiB

AXI private region · 256 KiB

0x24000000 → 0x24040000

Static data and ordinary heap.
Actual heap capacity: 234,336 B.

Thread stacks use RAM too

Most workers allocate stacks from their core's heap. The planner has a static DTCM stack.

An 8 KiB interrupt stack is not a thread pool.

Shared / external RAM

SRAM4 · 64 KiB

0x38000000 · ICC reserve: 60 KiB
49,312 B used; final 4 KiB: boot handoff

External SDRAM · 8 MiB

Framebuffers · 4 MiB reserved

0xC0000000 · non-cacheable
Two images, 1,228,800 B each

M4 ordinary code · 2 MiB

0xC0400000 · 600,984 B linked
Enabled in this flashed test configuration

UI working arena · 2 MiB

0xC0600000 · cacheable
3D colour/depth buffers: 1,500 KiB
Planning arena: 256 KiB

Spare SDRAM does not automatically enlarge either private heap.

Internal flash · 2 MiB

M7 bootstrap: 0x08000000 · 128 KiB
M4 bootstrap: 0x08100000 · 128 KiB
File storage: 0x08180000 · 512 KiB

Bootstrap initializes memory and copies RAM sections before starting both applications.

External QSPI flash · 16 MiB · images survive power-off

1	M4 · 3 MiB	M7 · 4 MiB	Assets · 6 MiB	1	1

0x90000000 · Factory area · 1 MiB
0x90100000 · M4 image · 3 MiB
0x90400000 · M7 image · 4 MiB
0x90800000 · Future assets · 6 MiB
0x90E00000 · Manifest / scratch · 1 MiB
0x90F00000 · Factory area · 1 MiB

Sources: cmake/application_giga.cmake; boot linker/manifest; build30 linked sizes. Heap capacity excludes linker/runtime metadata.

Which functions run from faster memory?

Execution addresses in the last flashed image; QSPI also holds the load copies of code that runs in RAM.

M4 · execution and communication

Internal SRAM · fast code region

`0x10000800 · 192 KiB reserved`

STEP generation
StepGenerator, StepHardware, StepOutput
Motion evaluation
Executor, ProfileSchedule, Profile, Sequence
Motor / safety paths
StepperMotor, feedback, driver fault handling
Communication paths
Bridge, ServiceBridge, ActionBridge, transport
Runtime / helpers
ThreadX, clock, notification, selected libc/libgcc

195,280 / 196,608 B · 99.32% · only 1,328 B remain

External SDRAM · ordinary M4 code

`0xC0400000 · 600,984 B linked`

Server dispatch, admission code and remaining application helpers run here in the flashed test. This avoids their QSPI instruction fetches; it is not equivalent to internal SRAM.

QSPI · remaining read-only material

Ordinary constants generally remain in flash; constants selected with fast code move to internal SRAM.

M7 · presentation and planning

ITCM · selected TGX renderer routines

`0x00000000 · 16,088 B linked`

`_drawMesh uber_shader _discardBox`

Selected geometry / raster paths avoid external instruction fetches. The current allocation is below the 24 KiB renderer cap.

QSPI + M7 instruction cache

`Ordinary application starts at 0x90400000`

GUI, CLI, Robot and Planner code execute here. The M7 caches instructions; the M4 does not have an equivalent instruction cache.

DTCM · planning stack and CPU data

The planning worker has a static 24 KiB DTCM stack. Its large planning arena is in SDRAM.

A fast stack does not place the planner’s functions in ITCM.

Important configuration distinction

Last flashed test: GIGA_M4_SDRAM_CODE=ON. Source default: OFF, so ordinary M4 code runs from QSPI unless enabled. Selected RAM entry points do not prove that every helper or indirect call also executes from RAM.

Sources: cmake/giga_fast_code.cmake; application_giga.cmake; linked ELF symbols / manifest; CMake cache for flashed test pair.

What is actually stopping us now?

The latest GIGA workflow fails while creating the next homing worker. The exact resource failure is not isolated.

1 Job prepared

IO accepted preparation.
Preparation took 8 ms.

→

2 M3 homed

Seek, back off, re-latch.
Reference was established.

→

3 M1 worker failed

“No more processes”
Thread creation returned EAGAIN.

→

4 Robot job faulted

Sequence stopped.
M2 was not reached.

Motors were disconnected; reference inputs were simulated. This is software / electrical evidence, not mechanical homing.

Observed pressure: M4 runtime heap

Snapshot taken after the failure · capacity 248,920 B

224,904 B used · 90.4%

24,016 B free in total

15 free blocks · largest usable block was not reported

Starting a homing worker requires

16 KiB stack + **Control / TLS allocations**

Total free memory alone does not establish whether this allocation sequence could succeed at the instant of failure.

Known vs. still unproven

KNOWN

The full robot homing sequence is broken on GIGA. Homing workers are created dynamically; EAGAIN can come from several runtime creation paths.

NOT ESTABLISHED

Which allocation or ThreadX operation failed. Heap fragmentation is a hypothesis, not a confirmed diagnosis.

Internal code/data regions are also tight: M4 fast code is 99.32% full; static data/DMA is 96.11% full.

Linux · passed

Motion API workflow passed.
72 additional regression tests passed.

GIGA · partial

M3 homing completed.
Full robot sequence failed.

Not yet qualified

Sustained 3-axis application timing,
full recovery and powered mechanics.

Current state: not a working, qualified motion baseline. Simulation success does not close the hardware failure.